Exercise 02
SQL Aggregate Functions and Operations

```
39              --Question 4--
40              --Get all students whose age is between 21 and 23.
41              SELECT*
42          ····FROM School.Subject.Students
43          ····WHERE Age BETWEEN 21 AND 23;
44
45                  --Question 5--
46
47                 --List all students in the IT or HR department who are older than 21
```

Add parameter

Table

	Student_id	Name	Age	Department
1	2	Bob	22	HR
2	3	Charlie	21	IT
3	4	Diana	23	Finance
4	5	Eve	22	HR

```
45                  --Question 5--
46
47                  --List all students in the IT or HR department who are older than 21
48                  SELECT*
49          ····FROM School.Subject.Students
50          ····WHERE Department IN ('IT', 'HR') AND Age > 21
```

Add parameter

Table

	Student_id	Name	Age	Department
1	2	Bob	22	HR
2	5	Eve	22	HR

```sql
                --Question 5--

                --List all students in the IT or HR department who are older than 21
                SELECT*
                FROM School.Subject.Students
                WHERE Department IN ('IT', 'HR') AND Age > 21
```

Add parameter

Table

	Student_id	Name	Age	Department
1	2	Bob	22	HR
2	5	Eve	22	HR

```sql
            --question 3--
            --Show Departments with more than 1 student.
            SELECT Department, COUNT(*) AS Total_Students
            FROM School.Subject.Students
            GROUP BY Department
            HAVING COUNT(*) > 1;

               --Question 4--
               --Get all students whose age is between 21 and 23.
               SELECT*
               FROM School.Subject.Students
```

Add parameter

Table

	Department	Total_Students
1	IT	7
2	HR	12
3	Finance	6

```sql

        --Question 1--
        --List all distinct departments in the table.
        SELECT DISTINCT Department
    FROM School.Subject.Students;

            --Question 2--
            --Get the average age of students per department.

```

Add parameter

Table

	Department
1	IT
2	HR
3	Finance

```sql
--Question 2--
--Get the average age of students per department.

SELECT Department, AVG(Age) AS Avg_Age
FROM School.Subject.Students
GROUP BY Department;

--question 3--
--Show Departments with more than 1 student.
SELECT Department, COUNT(*) AS Total_Students
FROM School.Subject.Students
GROUP BY Department
HAVING COUNT(*) > 1;
```

Add parameter

Table

	Department	Avg_Age
1	IT	20.8571428571428...
2	HR	22
3	Finance	23

```sql
--question 5
---List all project with budget between 50,000 and 120000,exclude the marketing department
SELECT
  project_id,
  project_name,
  department,
  budget
  FROM
  Interwaste.Construction.Projects
WHERE
  budget BETWEEN 50000 AND 120000

```

Add parameter

Table

	project_id	project_name	department	budget
1	1	AI App	IT	120000
2	2	Payroll Systems	Finance	80000
3	5	HR Portal	HR	50000

```sql

            --QUESTION 1--
  --SHOW TOTAL CREDITS PER DEPARTMENT,ONLY FOR DEPARTMENT WITH MORE THAN 5 TOTAL CREDITS.
  --EXPECTED COLUMNS: DEPARTMENT, TOTAL_CREDITS
SELECT Department, SUM(Credits) AS TOTAL_CREDITS
FROM College.students.Courses
GROUP BY Department
HAVING SUM(Credits) > 5;
```

Add parameter

Results 6 of 6

Table

	Department	TOTAL_CREDITS
1	IT	42
2	Finance	12
3	HR	18
4	it	24

```sql
-- Question 2
--count how many enrollment exist per course.
--expected columns: Course_id, Enrollment_Count
SELECT Course_id, COUNT(*) AS Enrollment_Count
FROM University.Courses.Enrollments
GROUP BY Course_id;
```

Add parameter

Results 7 of 7 | Table

	Course_id	Enrollment_Count
1	105	4
2	104	4
3	103	4
4	102	4

5 rows | 6.166s runtime | See performance (7) | Checked for improvements

```sql
-- Question 1
--Get the Maximum, Minimum, and Average Grade across all enrollment.
--excpected columns: Max_Grade, Min_Grade, Avg_Grade
SELECT MAX(Grade) AS Max_Grade, MIN(Grade) AS Min_Grade, AVG(Grade) AS Avg_Grade
FROM University.Courses.Enrollments
```

Add parameter

Table

	Max_Grade	Min_Grade	Avg_Grade
1	90	78	84.6

```sql
                --QUESTION 3--
--show the top 3 courses by credits in descending order
--Expected columns: Course_id, Course_Name, Department, Credits
SELECT*
FROM College.students.Courses
ORDER BY Credits DESC
LIMIT 3;
```

Add parameter

Table

	Course_id	Course_Name	Department	Credits
1	102	Python	it	4
2	103	Data Science	IT	4
3	102	Python	it	4

```sql
--show toal and average budget per department,only include departments with average budget > 70,000
SELECT
  department,
  sum(budget) as total_budget,
  AVG(budget) as average_budget
FROM
  Interwaste.Construction.Projects
GROUP BY
```

Add parameter

Results 6 of 6

Table

	department	total_budget	average_budget
1	IT	810000	135000
2	Finance	240000	80000

```
16                  --QUESTION 2--
17    --List all courses that do not have 4 credits
18    --Expected columns: Course_id, Course_Name, Department, Credits
19  SELECT*
20  FROM College.students.Courses
21  WHERE Credits <> 4;
22
23                  --QUESTION 3--
```

Add parameter

Table

	Course_id	Course_Name	Department	Credits
1	101	Sql Basics	IT	3
2	104	EXCEL	Finance	2
3	105	Statistics	HR	3
4	101	Sql Basics	IT	3

```sql
--QUESTION 4
--SHOW DEPARTMENT WHERE AVERAGE IS ABOVE 55000
SELECT
  Deparment,
  AVG(Salary) AS Avg_Salary
FROM Company.Department.Salaries
GROUP BY Deparment
HAVING Avg_Salary > 55000;
```

Add parameter

Table

	Deparment	Avg_Salary
1	IT	61000
2	Finance	70000

```sql
--QUESTION 4
-- LIST EMPLOYEES WHOSE SALARY PLUS BONUS IS ABOVE 60000
SELECT
 Employee_id,
 Name,
 Salary,
 Bonus,
 Salary + Bonus AS Total_Compesation
FROM Company.Department.Salaries
WHERE Salary + Bonus > 60000;
```

dd parameter

Results 8 of 8

Table

	Employee_id	Name	Salary	Bonus	Total_Compesation
1	1	Tom	60000	5000	65000
2	3	Spike	70000	6000	76000
3	4	Tyke	62000	5500	67500